Trabalho 1 — Concorrência e Sincronização: O Problema dos Filósofos Jantando

Grupo: Lorenzo Kalil · Mateus Huster · [3º integrante a definir] | **Disciplina:** Fundamentos de Programação Paralela e Distribuída (PUCRS) | **Código-fonte (Git):** https://github.com/llkalil/T1_FPPD

Soluções, mecanismos de sincronização e como são implementados em Go

Cinco (ou N) filósofos, um garfo entre cada par. O filósofo *i* usa os garfos *i* (esquerdo) e *(i+1) mod N* (direito). Cada garfo é um `sync.Mutex` (`Lock`/`Unlock`: exclusão mútua, sem preempção e não reentrante); cada filósofo é uma goroutine e `sync.WaitGroup` aguarda o fim. Pensar/comer é `time.Sleep` (10 ms cada), igual em todas as versões, e as métricas usam `sync/atomic` (contador de filósofos comendo, com máximo via CAS) e medição por filósofo do tempo entre o fim de pensar e a posse dos dois garfos. Um *watchdog* declara deadlock se nenhum ciclo termina por 5 s e imprime quem segura e quem espera cada garfo (o detector nativo do runtime não dispara porque há timers ativos).

- **V0 — sequencial (baseline):** um filósofo por vez, sem goroutines nem garfos; o tempo esperado é N × 20 × (10+10) ms. Foi escrita junto com as demais versões no mesmo programa, com a mesma instrumentação de métricas, em vez de adaptar o código de referência do Moodle: assim o tempo medido sai do mesmo código e a comparação é direta.
- **V1 — deadlock:** todos pegam primeiro o garfo esquerdo e depois o direito, com pausa de 1 ms entre os dois (`-gap`, só nesta versão) para tornar o deadlock provável. Se todos seguram o esquerdo, ninguém avança.
- **V2 — sincronização excessiva:** um mutex global é adquirido antes dos garfos e só liberado depois de comer. Correta e sem deadlock, mas serializa a mesa inteira: só um filósofo come por vez, mesmo havendo garfos livres.
- **V3 — aprimorada:** *hierarquia de recursos* [1]: cada filósofo pega sempre o garfo de menor índice primeiro. O último filósofo (N−1) inverte a ordem (garfo 0 antes do N−1), o que quebra a espera circular; vizinhos não conflitantes comem em paralelo.

V1: por que ocorre deadlock (as 4 condições de Coffman [2])

Exclusão mútua: um garfo (Mutex) só tem um dono. **Segura-e-espera:** cada filósofo segura o garfo *i* e espera o *i+1*. **Ausência de preempção:** o Mutex não pode ser tomado à força. **Espera circular:** 0→1→2→3→4→0 (o filósofo *i* espera o garfo em posse do *i+1*). Com as quatro presentes o sistema trava: cada filósofo tem **1 garfo** (o esquerdo) e espera **1** (o direito, do vizinho); ver captura na página 2.

Como V2 e V3 evitam o deadlock; starvation

V2 nega *segura-e-espera/espera circular*: só quem tem o mutex global pede garfos, e como ninguém mais os segura eles sempre estão livres. **V3** nega a *espera circular*: ordenando os recursos, a cadeia de esperas é sempre crescente e não fecha um ciclo. **Starvation:** em V2 e V3 o `sync.Mutex` do Go [4] entra em modo de inanição após 1 ms de espera (entrega direta em FIFO ao mais antigo), o que limita a espera de cada goroutine; observamos todos os filósofos concluindo os 20 ciclos em todas as execuções. Em teoria a V3 sozinha, com escalonamento arbitrário, não garante justiça; aqui ela vem do mutex.

Ambiente de execução

AWS EC2 **c8g.16xlarge** (spot, us-east-1b): AWS Graviton4, **64 vCPUs** (1 thread/core), Amazon Linux 2023 (kernel 6.18, aarch64), **Go 1.26.8 linux/arm64**. Compilação: `go build -o filosofos .` · Execução: `NS="5 48 96" RUNS=5 ./run.sh` (V1: 10 execuções por N; individual: `./filosofos -v 3 -n 96 -csv`). 20 ciclos por filósofo, pensar = comer = 10 ms, 5 execuções por versão e N (o enunciado pede N=5; N=48 e 96 foram acrescentados para ver a escala). Como pensar/comer são *sleep*, o resultado não depende do número de cores (tentamos a c8g.48xlarge/24xlarge, sem capacidade spot).

Visão geral das versões

Versão	Sincronização	Deadlock?	Máx. comendo (N=5/48/96)	Total N=96
V0	nenhuma (sequencial)	não	1 / 1 / 1	38.9 s
V1	1 mutex por garfo, esq→dir	sim (30/30)	0	não termina
V2	mutex global + garfos	não	1 / 1 / 1	19.5 s
V3	garfos ordenados (menor índice 1º)	não	2 / 23 / 44	0.53 s

Análise comparativa (resumo)

V3 é a melhor: com N=96 leva 0.53 s contra 19.5 s da V2 (37× mais lenta) e 38.9 s da V0 (73×), com até 44 filósofos comendo em paralelo. V2 é correta, mas só ganha ~2.0× sobre a V0 (sobrepõe apenas o pensar) e a espera média cresce com N (29.769 → 927.872 ms). V1 travou em 30 de 30 execuções. Repetido na c8g.medium (1 vCPU), V0/V2/V1 deram o mesmo resultado e a V3 foi 17–18% mais rápida com N≥48: o número de vCPUs quase não importa (página 3).

Resultados experimentais

Versão	N	Exec. (ok/dead)	Total médio (ms)	Desvio (ms)	Mín–Máx (ms)	Espera média (ms)	Maior espera (ms)	Máx. comendo	Ciclos/ filósofo
V0	5	5/0	2025.3	0.4	2024.5–2025.5	0.000	0.003	1	20–20
V0	48	5/0	19435.0	0.2	19434.7–19435.2	0.000	0.000	1	20–20
V0	96	5/0	38870.2	0.5	38869.8–38871.0	0.000	0.000	1	20–20
V1	5	0/10	deadlock	-	-	-	-	-	parcial: 0
V1	48	0/10	deadlock	-	-	-	-	-	parcial: 0
V1	96	0/10	deadlock	-	-	-	-	-	parcial: 0
V2	5	5/0	1022.7	0.0	1022.7–1022.8	29.769	50.613	1	20–20
V2	48	5/0	9730.2	0.8	9729.6–9731.5	454.154	486.072	1	20–20
V2	96	5/0	19450.8	1.2	19448.8–19451.7	927.872	972.037	1	20–20
V3	5	5/0	534.7	8.4	526.6–546.8	5.003	20.370	2	20–20
V3	48	5/0	511.6	30.1	467.0–537.9	2.443	40.581	23	20–20
V3	96	5/0	532.8	15.4	508.4–549.4	3.335	50.828	44	20–20

Tabela 1 — Médias sobre as execuções concluídas (c8g.16xlarge, 20 ciclos, pensar=comer=10 ms). Dados crus: `c8g.16xlarge/results.csv`. Na V1 o "tempo" medido seria só o do watchdog (5,1 s) e não é comparável.

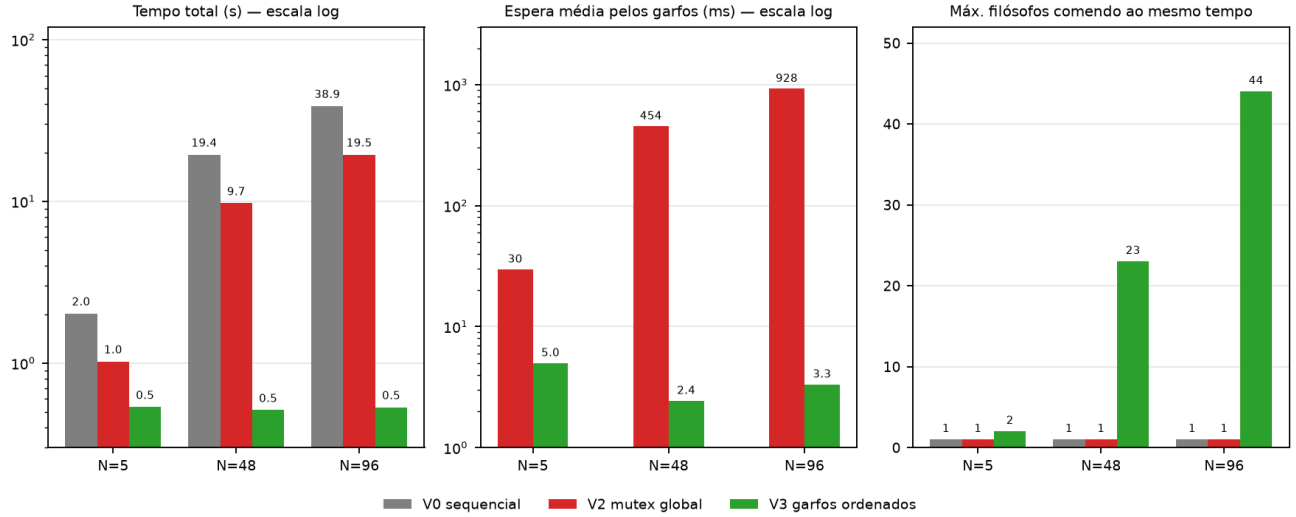


Figura 1 — Comparação por versão e N (barras: média das 5 execuções).

Captura 1 — V1 em deadlock (saída real do terminal, `./filosofos -v 1 -n 5`)

```
$ ./filosofos -v 1 -n 5
V1 n=5 status=DEADLOCK
  tempo total:        5100.51 ms
  espera média:       0.000 ms
  maior espera:       0.000 ms
  máx. comendo ao mesmo tempo: 0
  ciclos por filósofo: 0 0 0 0 0

== Evidência de deadlock ==
filósofo 0: tem garfo 0, ESPERA garfo 1 (em posse do filósofo 1) | ciclos=0
filósofo 1: tem garfo 1, ESPERA garfo 2 (em posse do filósofo 2) | ciclos=0
filósofo 2: tem garfo 2, ESPERA garfo 3 (em posse do filósofo 3) | ciclos=0
filósofo 3: tem garfo 3, ESPERA garfo 4 (em posse do filósofo 4) | ciclos=0
filósofo 4: tem garfo 4, ESPERA garfo 0 (em posse do filósofo 0) | ciclos=0
```

Captura 2 — linhas reais do `results.csv` (versao, n, status, total_ms, espera_media_ms, espera_max_ms, max_comendo, ciclos)

```
V2,96,OK,19451.67,927.932,971.976,1,20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;...
V3,96,OK,549.36,2.283,50.828,44,20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;20;2...
```

Conclusão e análise dos resultados

1. Como funciona cada solução. V0 executa os filósofos em sequência (baseline). V1 pega esquerdo e depois direito e trava. V2 envolve pegar-comer-soltar num mutex global. V3 impõe ordem global de aquisição dos garfos (menor índice primeiro).

2. Mecanismos. Exclusão mútua por `sync.Mutex` (um por garfo, mais um global na V2), `WaitGroup` para juntar goroutines, `atomic` para contadores e um watchdog para detectar deadlock. Nenhum canal foi necessário.

3. Implementação em Go. Mutex do runtime (sem preempção, não reentrante) sobre semáforos; goroutines escalonadas em até 64 threads do SO (GOMAXPROCS = 64); `time.Sleep` simula pensar/comer sem consumir CPU, por isso o desempenho é dominado pela *estrutura de sincronização*, não pelo número de cores.

4. Deadlock na V1. As quatro condições de Coffman ocorrem ao mesmo tempo (página 1). A evidência coletada é o estado de cada filósofo no travamento: todos com 1 garfo (o esquerdo) esperando o direito, formando o ciclo 0→1→…→N−1→0, com 0 ciclos concluídos. Travou em **30 de 30** execuções (N=5, 48 e 96). O *gap* de 1 ms entre os dois garfos alinha os filósofos e torna isso quase certo; em testes locais preliminares (outra máquina, sem controle) algumas execuções chegaram a terminar, ou seja, o deadlock depende de timing. Sem o gap não medimos a probabilidade.

5. Como V2 e V3 evitam. V2 impede *segura-e-espera* e a espera circular (um só filósofo por vez disputa garfos). V3 impede a espera circular pela ordenação: como o último filósofo pega o garfo 0 antes do N−1, não existe ciclo de dependências. Ambas concluíram 100% das execuções (V2: 15/15, V3: 15/15).

6. Starvation. Em teoria a V3 usa só ordenação e não garante justiça, e a V2 depende da fila do mutex global. Na prática o modo de inanição do `sync.Mutex` (1 ms) faz a espera ser limitada. Medimos isso: todos os filósofos fizeram exatamente 20 ciclos em todas as execuções, e a maior espera foi 972.037 ms na V2 (≈ N×10 ms = 960 ms: cada um espera a fila inteira comer) e 50.828 ms na V3 (~5 tempos de comer). Não observamos starvation; a V1 não se aplica, pois trava antes.

7. Maior tempo total. Entre as que terminam, a **V0** (N=96: 38.9 s; N=48: 19.4 s; N=5: 2.03 s), que coincide com o previsto N×20×20 ms. A V1 não termina (tempo infinito).

8. Maior tempo de espera. A **V2**: média de 29.769 ms (N=5), 454.154 ms (N=48) e 927.872 ms (N=96), crescendo linearmente com N, contra 5.003–3.335 ms na V3 e ~0 na V0 (não há disputa). Na V1 a espera é infinita.

9. Filósofos comendo simultaneamente. V0: 1. V2: 1 (mesmo com garfos livres). V3: 2 (N=5), 23 (N=48) e 44 (N=96), próximo do teto teórico N/2 = 2, 24 e 48 (arredondando para baixo). V1: 0 (ninguém chega a comer).

10. Impacto da sincronização excessiva (V2). A V2 só é ~2.0× melhor que a V0: o mutex global já permite sobrepor o *pensar* de uns com o comer de outro, mas serializa o comer. Frente à V3 ela é 1.9× mais lenta com N=5, 19× com N=48 e 37× com N=96 — o custo cresce com N porque a V3 mantém o tempo quase constante e a V2 cresce linearmente.

11. Mais sincronização = melhor desempenho? Não. A V2 é a que mais sincroniza (mutex global *além* dos garfos) e foi a pior das corretas, com maior espera e concorrência 1. A V3 usa a *mesma* quantidade de locks de garfo, mas sem o ponto de serialização, e teve o menor tempo total e a menor espera. Importa *onde* se sincroniza (escopo da seção crítica), não quanto. Também não é mais sincronização o que evita deadlock, e sim a estrutura (ordem) dela.

12. Grau de concorrência × tempo total. O tempo é aproximadamente T ≈ (trabalho a serializar) ÷ (concorrência efetiva): V0 (concorrência 1): T = N×20×20 ms; V2 (o comer é serial, o pensar sobrepõe): T ≈ N×20×10 ms (medido 19.45 s vs 19,2 s previstos com N=96); V3 (concorrência ≈ N/2): T ≈ 20×(pensar+comer) = 400 ms, medido ≈ 533 ms (+30%, por disputa de garfos vizinhos e jitter de timers). Speedup da V3 sobre a V0: 3.8× (N=5), 38× (N=48), 73× (N=96), acompanhando o aumento da concorrência observada (2 → 23 → 44).

Comparação: c8g.medium (1 vCPU) × c8g.16xlarge (64 vCPUs)

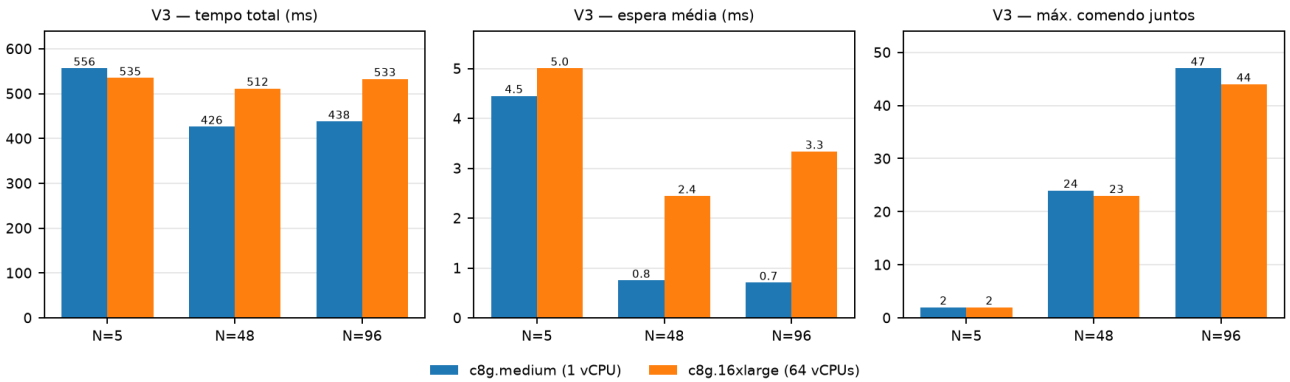


Figura 2 — Efeito do número de vCPUs na V3 (V0, V2 e V1 não mudam: ver tabela). Barras: média de 5 execuções.

Versão	N	Total (ms) medium	Total (ms) 16xlarge	Espera média (ms) medium / 16xl	Máx. comendo medium / 16xl
V0	5	2027.7	2025.3	0.000 / 0.000	1 / 1
V0	48	19419.3	19435.0	0.000 / 0.000	1 / 1
V0	96	38838.6	38870.2	0.000 / 0.000	1 / 1
V2	5	1021.5	1022.7	29.834 / 29.769	1 / 1
V2	48	9721.7	9730.2	453.961 / 454.154	1 / 1
V2	96	19431.4	19450.8	927.301 / 927.872	1 / 1
V3	5	556.4	534.7	4.451 / 5.003	2 / 2
V3	48	426.2	511.6	0.750 / 2.443	24 / 23
V3	96	437.8	532.8	0.705 / 3.335	47 / 44

A medium (1 vCPU, GOMAXPROCS=1) repetiu o mesmo experimento (5 execuções por caso; V1 com 10). **V0 e V2 ficaram idênticos** (razão 0,999–1,001): são limitados por *sleep* e por serialização, não por CPU. **A V1 travou nas 30 execuções também na medium**: o deadlock é lógico e não depende de núcleos. **A V3 foi ~17–18% mais rápida na medium** com N=48 e 96 (e ~4% mais lenta com N=5) (437.8 ms contra 532.8 ms com N=96), com espera média menor (0.705 contra 3.335 ms), desvio-padrão quase zero (0.1 ms contra 15.4 ms) e concorrência máxima de 47 filósofos (teto 48). Confirmado na seção seguinte (a mesma 16xlarge com GOMAXPROCS=1 reproduz a medium): com um só P, os timers acordam os filósofos em lotes alinhados e há menos migração/jitter entre threads; na 16xlarge, o escalonamento em 64 threads adiciona atrasos (mais variação). Conclusão: para esta carga, mais cores **não** melhoram o desempenho (acima de 1 P há uma piora única de ~15–20% e depois um patamar estável); a estrutura de sincronização é o fator dominante (V2 → V3 = 19–37×), e um núcleo basta.

Isolando o efeito dos cores: GOMAXPROCS na mesma c8g.16xlarge

Para separar "número de cores" de "máquina diferente", repetimos a V3 (N=48 e 96, 5 execuções) e a V2 (N=48, 3 execuções) na **mesma** 16xlarge variando `GOMAXPROCS` (1, 2, 4, 8, 16, 64). Script: `filosofos/run_gmp.sh`; dados: `gomaxprocs/gmp.csv`.

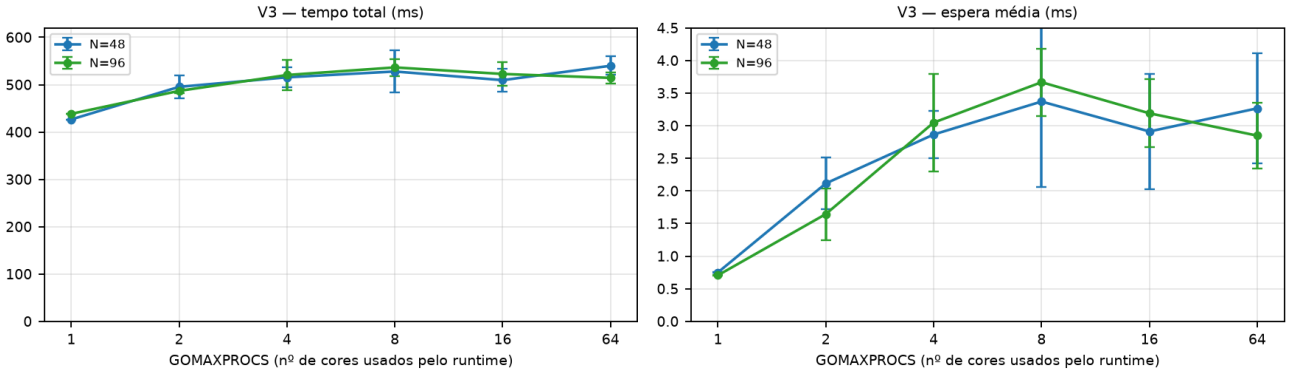


Figura 3 — V3: tempo total e espera média × GOMAXPROCS (média ± desvio-padrão).

GOMAXPROCS	V3 N=48 total (ms) ± dp	V3 N=48 espera (ms)	V3 N=96 total (ms) ± dp	V3 N=96 espera (ms)	V2 N=48 total (ms)
1	426.6 ± 0.0	0.75	438.4 ± 0.1	0.71	9722
2	495.4 ± 24.2	2.12	486.9 ± 4.5	1.65	9727
4	515.5 ± 20.8	2.86	520.4 ± 31.6	3.05	9726
8	527.9 ± 44.7	3.37	536.5 ± 18.1	3.67	9727
16	509.5 ± 24.2	2.91	522.7 ± 25.3	3.19	9727
64	539.9 ± 19.5	3.27	514.4 ± 11.5	2.85	9729

Hipótese confirmada: com GOMAXPROCS=1 a 16xlarge dá 426.6 ms (dp 0.0) com N=48, praticamente igual à c8g.medium (426.2 ms). Portanto a divergência **não vem da máquina**, e sim do número de Ps (processadores lógicos do escalonador do Go). Já com 2 Ps o tempo sobe para 495 ms e, de 4 até 64, fica num patamar de ~490–540 ms com desvio de 5–45 ms: **acima de 1 P o desempenho piora (~15–20%) para um patamar estável, sem melhora adicional com mais cores**. As diferenças entre 4, 8, 16 e 64 Ps cabem no ruído de 5 execuções e não indicam piora progressiva. Com um só P os timers e o escalonamento são determinísticos e os filósofos ficam em fase; com vários Ps os acordares se espalham e a fase muda a cada execução, aumentando a disputa entre vizinhos. A causa exata dentro do escalonador não foi investigada (isso exigiria *tracing*). A V2 é insensível: 9722 ms com 1 P e 9729 ms com 64 (o mutex global já serializa).

Referências

[1] DIJKSTRA, E. W. Hierarchical ordering of sequential processes. *Acta Informatica*, v. 1, n. 2, p. 115–138, 1971. (problema dos filósofos e ordenação de recursos, usada na V3)

[2] COFFMAN, E. G.; ELPHICK, M. J.; SHOSHANI, A. System deadlocks. *ACM Computing Surveys*, v. 3, n. 2, p. 67–78, 1971. (as quatro condições de deadlock)

[3] CHANDY, K. M.; MISRA, J. The drinking philosophers problem. *ACM TOPLAS*, v. 6, n. 4, p. 632–646, 1984. (solução distribuída alternativa, não implementada aqui)

[4] THE GO AUTHORS. Package sync (Mutex, WaitGroup) e fonte `sync/mutex.go`, que documenta o modo normal e o modo de inanição. https://pkg.go.dev/sync

[5] THE GO AUTHORS. Packages `sync/atomic`, `time` e `runtime` (GOMAXPROCS). https://pkg.go.dev

[6] TANENBAUM, A. S.; BOS, H. *Modern Operating Systems*. Pearson. (filósofos jantando e deadlock); SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. Wiley.

[7] PUCRS. Trabalho 1 — Concorrência e Sincronização: o problema dos filósofos jantando (adaptação do Prof. De Rose). Enunciado e material de apoio (Moodle).

Ressalvas

- O enunciado usa N=5; N=48 e 96 foram acrescentados para evidenciar a escala. Em N=5 o ganho da V3 sobre a V2 é modesto (1,9×) e a V3 é limitada a 2 comensais.
- Pensar/comer são simulados com *sleep*; com trabalho real de CPU, o número de cores passaria a limitar a V3. Desvios-padrão baixos (≤ 31 ms) indicam medidas estáveis; 5 execuções por caso.
- Medidas de espera incluem a espera pelo mutex global (V2) e pelos dois garfos (V1/V3). A V1 é analisada à parte: as 30 execuções terminaram em deadlock, com evidência em `deadlock_n5/48/96.txt`.
- As “capturas” são a saída textual real do terminal; código completo no repositório indicado na página 1.

Apêndice A — Ambiente completo

c8g.16xlarge

```
data: 2026-09-28T23:27:43Z
go version go1.26.8-X:nodwarf5 linux/arm64
Linux 6.18.48-109.150.amzn2023.aarch64 aarch64
CPU(s):                                  64
Thread(s) per core:                      1
Core(s) per socket:                      64
Socket(s):                               1
GOMAXPROCS padrão = nproc = 64
compilar: go build -o filosofos .
executar: NS="5 48 96" RUNS=5 ./run.sh
```

c8g.medium

```
data: 2026-09-29T00:00:11Z
go version go1.26.8-X:nodwarf5 linux/arm64
Linux 6.18.48-109.150.amzn2023.aarch64 aarch64
CPU(s):                                  1
Thread(s) per core:                      1
Core(s) per socket:                      1
Socket(s):                               1
GOMAXPROCS padrão = nproc = 1
compilar: go build -o filosofos .
executar: NS="5 48 96" RUNS=5 ./run.sh
```

Apêndice B — Todas as execuções (dados crus)

Colunas: versão, N, status, total (ms), espera média (ms), maior espera (ms), máx. comendo, ciclos por filósofo (“20x96” = todos os 96 filósofos com 20 ciclos; na V1 os ciclos são os do momento do deadlock). Arquivos originais: `results.csv` de cada pasta.

c8g.16xlarge (c8g.16xlarge / results.csv)

```
versao,n,status,total_ms,espera_media_ms,espera_max_ms,max_comendo,ciclos
V0,5,OK,2025.53,0.000,0.000,1,20x5
V0,5,OK,2025.52,0.000,0.003,1,20x5
V0,5,OK,2025.52,0.000,0.000,1,20x5
V0,5,OK,2025.18,0.000,0.000,1,20x5
V0,5,OK,2024.55,0.000,0.000,1,20x5
V1,5,DEADLOCK,5100.50,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.49,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.42,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.42,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.52,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.42,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.42,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.50,0.000,0.000,0,0x5
V1,5,DEADLOCK,5200.52,0.000,0.000,0,0x5
V1,5,DEADLOCK,5200.35,0.000,0.000,0,0x5
V2,5,OK,1022.75,29.870,40.492,1,20x5
V2,5,OK,1022.71,29.668,50.597,1,20x5
V2,5,OK,1022.74,29.869,40.496,1,20x5
V2,5,OK,1022.77,29.568,50.613,1,20x5
V2,5,OK,1022.74,29.870,40.514,1,20x5
V3,5,OK,546.82,4.860,20.345,2,20x5
V3,5,OK,526.68,5.064,20.249,2,20x5
V3,5,OK,526.61,5.062,20.348,2,20x5
V3,5,OK,536.83,4.964,20.370,2,20x5
V3,5,OK,536.79,5.063,20.358,2,20x5
V0,48,OK,19434.90,0.000,0.000,1,20x48
V0,48,OK,19434.67,0.000,0.000,1,20x48
V0,48,OK,19435.11,0.000,0.000,1,20x48
V0,48,OK,19435.24,0.000,0.000,1,20x48
V0,48,OK,19435.17,0.000,0.000,1,20x48
V1,48,DEADLOCK,5100.50,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.59,0.000,0.000,0,0x48
V1,48,DEADLOCK,5200.51,0.000,0.000,0,0x48
V1,48,DEADLOCK,5200.56,0.000,0.000,0,0x48
V1,48,DEADLOCK,5200.51,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.66,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.50,0.000,0.000,0,0x48
V1,48,DEADLOCK,5200.66,0.000,0.000,0,0x48
V1,48,DEADLOCK,5200.38,0.000,0.000,0,0x48
V1,48,DEADLOCK,5200.41,0.000,0.000,0,0x48
V2,48,OK,9731.54,453.938,486.072,1,20x48
V2,48,OK,9729.58,454.328,475.802,1,20x48
V2,48,OK,9729.83,454.287,475.890,1,20x48
V2,48,OK,9730.28,453.877,485.922,1,20x48
V2,48,OK,9729.87,454.342,475.813,1,20x48
V3,48,OK,517.91,2.640,30.501,22,20x48
V3,48,OK,537.89,2.417,40.581,23,20x48
V3,48,OK,537.94,2.954,30.448,22,20x48
V3,48,OK,467.04,1.435,20.378,23,20x48
V3,48,OK,497.37,2.767,30.432,23,20x48
V0,96,OK,38870.96,0.000,0.000,1,20x96
V0,96,OK,38870.15,0.000,0.000,1,20x96
V0,96,OK,38869.85,0.000,0.000,1,20x96
V0,96,OK,38870.39,0.000,0.000,1,20x96
V0,96,OK,38869.80,0.000,0.000,1,20x96
V1,96,DEADLOCK,5200.82,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.66,0.000,0.000,0,0x96
V1,96,DEADLOCK,5200.50,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.69,0.000,0.000,0,0x96
```

```
V1,96,DEADLOCK,5200.73,0.000,0.000,0,0x96
V1,96,DEADLOCK,5200.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5200.57,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.67,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V2,96,OK,19451.67,927.932,971.976,1,20x96
V2,96,OK,19448.83,928.136,961.774,1,20x96
V2,96,OK,19450.32,927.715,972.037,1,20x96
V2,96,OK,19451.58,927.801,971.967,1,20x96
V2,96,OK,19451.51,927.776,971.988,1,20x96
V3,96,OK,549.36,2.283,50.828,44,20x96
V3,96,OK,508.42,3.184,30.419,43,20x96
V3,96,OK,538.69,3.152,50.778,43,20x96
V3,96,OK,538.70,4.158,30.510,41,20x96
V3,96,OK,528.77,3.900,30.519,41,20x96
```

c8g.medium (c8g.medium / results.csv)

```
versao,n,status,total_ms,espera_media_ms,espera_max_ms,max_comendo,ciclos
V0,5,OK,2034.80,0.000,0.003,1,20x5
V0,5,OK,2035.47,0.000,0.000,1,20x5
V0,5,OK,2022.68,0.000,0.000,1,20x5
V0,5,OK,2022.78,0.000,0.000,1,20x5
V0,5,OK,2022.78,0.000,0.000,1,20x5
V1,5,DEADLOCK,5100.52,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.68,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.53,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.52,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.53,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.52,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.52,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.53,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.52,0.000,0.000,0,0x5
V1,5,DEADLOCK,5100.52,0.000,0.000,0,0x5
V2,5,OK,1021.52,29.834,40.435,1,20x5
V2,5,OK,1021.47,29.833,40.433,1,20x5
V2,5,OK,1021.58,29.836,40.433,1,20x5
V2,5,OK,1021.49,29.833,40.434,1,20x5
V2,5,OK,1021.51,29.834,40.433,1,20x5
V3,5,OK,556.39,4.451,30.338,2,20x5
V3,5,OK,556.39,4.451,30.335,2,20x5
V3,5,OK,556.46,4.451,30.331,2,20x5
V3,5,OK,556.44,4.451,30.332,2,20x5
V3,5,OK,556.38,4.451,30.332,2,20x5
V0,48,OK,19419.67,0.000,0.000,1,20x48
V0,48,OK,19418.67,0.000,0.000,1,20x48
V0,48,OK,19419.08,0.000,0.000,1,20x48
V0,48,OK,19419.97,0.000,0.000,1,20x48
V0,48,OK,19418.90,0.000,0.000,1,20x48
V1,48,DEADLOCK,5100.68,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.84,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.68,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.68,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.52,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.84,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.68,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.68,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.68,0.000,0.000,0,0x48
V1,48,DEADLOCK,5100.68,0.000,0.000,0,0x48
V2,48,OK,9720.53,453.907,475.319,1,20x48
V2,48,OK,9720.50,453.908,475.298,1,20x48
V2,48,OK,9720.46,453.904,475.292,1,20x48
V2,48,OK,9721.68,453.962,475.210,1,20x48
V2,48,OK,9725.14,454.123,475.322,1,20x48
V3,48,OK,426.22,0.750,20.268,24,20x48
V3,48,OK,426.23,0.750,20.269,24,20x48
V3,48,OK,426.09,0.750,20.275,24,20x48
V3,48,OK,426.22,0.750,20.267,24,20x48
V3,48,OK,426.22,0.750,20.271,24,20x48
V0,96,OK,38838.06,0.000,0.000,1,20x96
V0,96,OK,38837.79,0.000,0.000,1,20x96
V0,96,OK,38839.94,0.000,0.000,1,20x96
V0,96,OK,38837.13,0.000,0.000,1,20x96
V0,96,OK,38840.07,0.000,0.000,1,20x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.83,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.83,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.83,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.68,0.000,0.000,0,0x96
V1,96,DEADLOCK,5100.98,0.000,0.000,0,0x96
V2,96,OK,19431.58,927.319,960.745,1,20x96
V2,96,OK,19433.03,927.341,960.820,1,20x96
V2,96,OK,19430.86,927.281,960.818,1,20x96
V2,96,OK,19430.86,927.282,960.771,1,20x96
V2,96,OK,19430.89,927.284,960.824,1,20x96
V3,96,OK,437.83,0.705,20.372,47,20x96
V3,96,OK,437.72,0.705,20.372,47,20x96
V3,96,OK,437.85,0.705,20.373,47,20x96
V3,96,OK,437.75,0.705,20.372,47,20x96
V3,96,OK,437.84,0.705,20.372,47,20x96
```

gomaxprocs (gomaxprocs / gmp.csv)

```
gomaxprocs,versao,n,status,total_ms,espera_media_ms,espera_max_ms,max_comendo,ciclos
1,V3,48,OK,426.63,0.752,20.284,24,20x48
1,V3,96,OK,438.39,0.706,20.374,47,20x96
1,V3,48,OK,426.53,0.751,20.283,24,20x48
1,V3,96,OK,438.38,0.706,20.397,47,20x96
1,V3,48,OK,426.60,0.751,20.284,24,20x48
1,V3,96,OK,438.36,0.706,20.403,47,20x96
```

```
1,V3,48,OK,426.55,0.751,20.283,24,20x48
1,V3,96,OK,438.51,0.707,20.389,47,20x96
1,V3,48,OK,426.55,0.751,20.282,24,20x48
1,V3,96,OK,438.60,0.708,20.375,47,20x96
1,V2,48,OK,9723.24,454.035,475.683,1,20x48
1,V2,48,OK,9721.72,453.964,475.476,1,20x48
1,V2,48,OK,9721.51,453.954,475.466,1,20x48
2,V3,48,OK,507.57,2.503,30.426,23,20x48
2,V3,96,OK,488.81,1.764,30.522,46,20x96
2,V3,48,OK,487.14,1.774,30.428,23,20x48
2,V3,96,OK,478.76,1.012,20.499,47,20x96
2,V3,48,OK,457.00,1.933,30.463,23,20x48
2,V3,96,OK,488.87,1.600,30.614,45,20x96
2,V3,48,OK,507.76,1.786,20.412,23,20x48
2,V3,96,OK,489.06,2.089,30.545,44,20x96
2,V3,48,OK,517.74,2.579,40.670,23,20x48
2,V3,96,OK,488.89,1.761,20.471,45,20x96
2,V2,48,OK,9726.98,453.719,475.821,1,20x48
2,V2,48,OK,9726.71,454.024,475.944,1,20x48
2,V2,48,OK,9726.82,453.946,475.646,1,20x48
4,V3,48,OK,497.12,2.678,30.313,23,20x48
4,V3,96,OK,568.95,4.294,40.589,42,20x96
4,V3,48,OK,517.68,2.533,30.404,23,20x48
4,V3,96,OK,498.30,2.477,30.466,44,20x96
4,V3,48,OK,517.63,2.764,30.407,23,20x48
4,V3,96,OK,487.73,2.483,30.418,44,20x96
4,V3,48,OK,497.29,2.879,30.385,23,20x48
4,V3,96,OK,518.49,3.121,40.612,42,20x96
4,V3,48,OK,547.83,3.471,30.420,22,20x48
4,V3,96,OK,528.70,2.866,30.433,43,20x96
4,V2,48,OK,9726.01,454.156,475.660,1,20x48
4,V2,48,OK,9725.79,454.149,475.668,1,20x48
4,V2,48,OK,9725.79,454.149,475.703,1,20x48
8,V3,48,OK,548.52,3.878,30.496,21,20x48
8,V3,96,OK,538.42,4.134,50.748,41,20x96
8,V3,48,OK,467.16,1.646,20.373,23,20x48
8,V3,96,OK,518.12,3.396,50.793,43,20x96
8,V3,48,OK,578.38,5.132,40.538,21,20x48
8,V3,96,OK,548.84,4.217,40.643,41,20x96
8,V3,48,OK,497.25,2.628,30.522,23,20x48
8,V3,96,OK,518.39,3.596,40.622,42,20x96
8,V3,48,OK,548.09,3.568,40.569,22,20x48
8,V3,96,OK,558.49,2.990,30.402,43,20x96
8,V2,48,OK,9725.76,454.148,475.624,1,20x48
8,V2,48,OK,9727.13,453.727,485.902,1,20x48
8,V2,48,OK,9726.76,453.707,485.816,1,20x48
16,V3,48,OK,547.86,2.078,30.497,24,20x48
16,V3,96,OK,508.65,2.952,30.464,43,20x96
16,V3,48,OK,518.01,4.025,30.404,21,20x48
16,V3,96,OK,559.12,3.890,40.765,42,20x96
16,V3,48,OK,497.38,3.400,40.568,22,20x48
16,V3,96,OK,538.52,3.489,30.726,44,20x96
16,V3,48,OK,487.29,1.963,40.604,23,20x48
16,V3,96,OK,498.50,3.097,20.458,42,20x96
16,V3,48,OK,497.16,3.092,20.354,21,20x48
16,V3,96,OK,508.49,2.523,50.781,44,20x96
16,V2,48,OK,9727.45,453.742,485.889,1,20x48
16,V2,48,OK,9726.20,454.167,475.702,1,20x48
16,V2,48,OK,9726.50,454.180,475.635,1,20x48
64,V3,48,OK,568.23,3.188,40.686,23,20x48
64,V3,96,OK,518.39,3.470,40.656,43,20x96
64,V3,48,OK,537.87,3.748,40.572,22,20x48
64,V3,96,OK,528.32,3.204,30.544,43,20x96
64,V3,48,OK,548.26,3.285,50.689,23,20x48
64,V3,96,OK,518.52,2.721,40.773,44,20x96
64,V3,48,OK,527.74,4.179,30.519,21,20x48
64,V3,96,OK,508.30,2.679,30.532,43,20x96
64,V3,48,OK,517.56,1.931,40.676,24,20x48
64,V3,96,OK,498.24,2.165,40.683,44,20x96
64,V2,48,OK,9728.81,454.288,475.763,1,20x48
64,V2,48,OK,9729.40,453.832,485.938,1,20x48
64,V2,48,OK,9729.14,453.967,475.997,1,20x48
```

Apêndice C — Evidências de deadlock (V1) em N=48 e N=96

N=48 (c8g.16xlarge) — 48 filósofos, todos com 1 garfo e esperando o do vizinho; primeiros 6 e último:

```
V1 n=48 status=DEADLOCK
  tempo total:        5100.50 ms
  espera média:       0.000 ms
  maior espera:       0.000 ms
  máx. comendo ao mesmo tempo: 0
  ciclos por filósofo: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

== Evidência de deadlock ==
filósofo 0: tem garfo 0, ESPERA garfo 1 (em posse do filósofo 1) | ciclos=0
filósofo 1: tem garfo 1, ESPERA garfo 2 (em posse do filósofo 2) | ciclos=0
filósofo 2: tem garfo 2, ESPERA garfo 3 (em posse do filósofo 3) | ciclos=0
filósofo 3: tem garfo 3, ESPERA garfo 4 (em posse do filósofo 4) | ciclos=0
filósofo 4: tem garfo 4, ESPERA garfo 5 (em posse do filósofo 5) | ciclos=0
filósofo 5: tem garfo 5, ESPERA garfo 6 (em posse do filósofo 6) | ciclos=0
...
filósofo 47: tem garfo 47, ESPERA garfo 0 (em posse do filósofo 0) | ciclos=0
```

N=96 (c8g.16xlarge) — 96 filósofos, todos com 1 garfo e esperando o do vizinho; primeiros 6 e último:

```
V1 n=96 status=DEADLOCK
  tempo total:        5200.54 ms
  espera média:       0.000 ms
  maior espera:       0.000 ms
  máx. comendo ao mesmo tempo: 0
  ciclos por filósofo: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
state := make([]int32, *n)
var progress, eating, maxEating atomic.Int64
ps := make([]phil, *n)

acquire := func(id int) {
    l, r := id, (id+1)%*n
    switch *ver {
    case 1:
        atomic.StoreInt32(&state[id], stWaitLeft)
        forks[l].Lock()
        atomic.StoreInt32(&state[id], stHoldLeftWaitRight)
        time.Sleep(*gap)
        forks[r].Lock()
    case 2:
        global.Lock() // segurado até o fim de comer: só 1 filósofo por vez
        forks[l].Lock()
        forks[r].Lock()
    case 3:
        if l > r { // sempre o garfo de menor índice primeiro: sem espera circular
            l, r = r, l
        }
        forks[l].Lock()
        forks[r].Lock()
    }
}
release := func(id int) {
    l, r := id, (id+1)%*n
    if *ver == 0 {
        return
    }
    forks[l].Unlock()
    forks[r].Unlock()
    if *ver == 2 {
        global.Unlock()
    }
}

worker := func(id int) {
    p := &ps[id]
    for c := 0; c < *cycles; c++ {
        atomic.StoreInt32(&state[id], stThinking)
        time.Sleep(*think)
        t0 := time.Now()
        if *ver != 0 {
            acquire(id)
        }
        w := time.Since(t0)
        atomic.StoreInt32(&state[id], stEating)
        cur := eating.Add(1)
        for {
            m := maxEating.Load()
            if cur <= m || maxEating.CompareAndSwap(m, cur) {
                break
            }
        }
        time.Sleep(*eat)
        eating.Add(-1)
        release(id)
        p.waitSum += w
        if w > p.waitMax {
            p.waitMax = w
        }
        p.cycles++
        progress.Add(1)
    }
}

done := make(chan struct{})
start := time.Now()
go func() {
    if *ver == 0 {
        for i := 0; i < *n; i++ {
            worker(i)
        }
    } else {
        var wg sync.WaitGroup
        for i := 0; i < *n; i++ {
            wg.Add(1)
            go func(i int) { defer wg.Done(); worker(i) }(i)
        }
        wg.Wait()
    }
    close(done)
}()

// watchdog: sem progresso por 'stall' => deadlock, imprime evidência
last, lastAt := int64(-1), time.Now()
tick := time.NewTicker(100 * time.Millisecond)
for running := true; running; {
    select {
    case <-done:
        running = false
    case <-tick.C:
        if p := progress.Load(); p != last {
            last, lastAt = p, time.Now()
        } else if time.Since(lastAt) > *stall {
            report(*csv, *ver, *n, "DEADLOCK", time.Since(start), ps, maxEating.Load())
            if !*csv {
                fmt.Println("\n== Evidência de deadlock ==")
                for i := 0; i < *n; i++ {
                    s := atomic.LoadInt32(&state[i])
                    fmt.Printf("filósofo %d: %s | ciclos=%d\n", i, describe(i, *n, s), ps[i].cycles)
                }
```

```
                }
                os.Exit(3)
            }
        }
    }
    report(*csv, *ver, *n, "OK", time.Since(start), ps, maxEating.Load())
}

func describe(i, n int, s int32) string {
    l, r := i, (i+1)%n
    switch s {
    case stHoldLeftWaitRight:
        return fmt.Sprintf("tem garfo %d, ESPERA garfo %d (em posse do filósofo %d)", l, r, r)
    case stWaitLeft:
        return fmt.Sprintf("esperando garfo %d", l)
    case stEating:
        return "comendo"
    }
    return "pensando"
}

func report(csv bool, ver, n int, status string, total time.Duration, ps []phil, maxEat int64) {
    var sum, mx time.Duration
    var cnt int
    cs := make([]string, len(ps))
    for i, p := range ps {
        sum += p.waitSum
        cnt += p.cycles
        if p.waitMax > mx {
            mx = p.waitMax
        }
        cs[i] = fmt.Sprint(p.cycles)
    }
    avg := time.Duration(0)
    if cnt > 0 {
        avg = sum / time.Duration(cnt)
    }
    ms := func(d time.Duration) float64 { return float64(d.Microseconds()) / 1000 }
    if csv {
        // versao,n,status,total_ms,espera_media_ms,espera_max_ms,max_comendo,ciclos_por_filosofo
        fmt.Printf("V%d,%d,%s,%.2f,%.3f,%.3f,%d,%s\n", ver, n, status, ms(total), ms(avg), ms(mx), maxEat, strings.Join(cs, ";"))
        return
    }
    fmt.Printf("V%d n=%d status=%s\n  tempo total:        %.2f ms\n  espera média:       %.3f ms\n  maior espera:       %.3f ms\n  máx. comendo ao
      mesmo tempo: %d\n  ciclos por filósofo: %s\n",
        ver, n, status, ms(total), ms(avg), ms(mx), maxEat, strings.Join(cs, " "))
}
```

filosofos/run.sh

```
#!/usr/bin/env bash
# Roda todas as versões RUNS vezes para cada N em NS. Saída: results.csv + env.txt
# Ex.: NS="5 48 96" RUNS=5 ./run.sh
set -u
cd "$(dirname "$0")"
NS=${NS:-"5 48 96"}
RUNS=${RUNS:-5}
V1_RUNS=${V1_RUNS:-10}   # V1 tenta mais vezes: precisa capturar deadlock
OUT=results.csv

go build -o filosofos . || exit 1

{
  echo "data: $(date -u +%FT%TZ)"
  go version
  uname -srm
  lscpu | grep -E 'Model name|^CPU\(s\)|Thread|Core|Socket'
  echo "GOMAXPROCS padrão = nproc = $(nproc)"
  echo "compilar: go build -o filosofos ."
  echo "executar: NS=\"$NS\" RUNS=$RUNS ./run.sh"
} > env.txt

echo "versao,n,status,total_ms,espera_media_ms,espera_max_ms,max_comendo,ciclos_por_filosofo" > $OUT
for n in $NS; do
  for v in 0 1 2 3; do
    r=$RUNS; [ $v = 1 ] && r=$V1_RUNS
    for i in $(seq $r); do
      ./filosofos -v $v -n $n -csv | tee -a $OUT
    done
  done
  # evidência legível de deadlock (última execução V1 que travar)
  for i in $(seq 20); do
    ./filosofos -v 1 -n $n > deadlock_n$n.txt && continue
    break
  done
done

echo; echo "== médias (execuções OK) =="
awk -F, 'NR>1 && $3=="OK"{k=$1" n="$2; c[k]++; t[k]+=$4; w[k]+=$5; if($6>m[k])m[k]=$6; if($7>e[k])e[k]=$7}
  NR>1 && $3=="DEADLOCK"{d[$1" n="$2]++}
  END{printf "%-10s %4s %10s %10s %10s %8s %5s\n","versao","runs","total_ms","espera_ms","espmax_ms","comendo","dead";
      for(k in c) printf "%-10s %4d %10.1f %10.3f %10.3f %8d %5d\n",k,c[k],t[k]/c[k],w[k]/c[k],m[k],e[k],d[k]+0;
      for(k in d) if(!(k in c)) printf "%-10s %4d %10s %10s %10s %8s %5d\n",k,0,"-","-","-","-",d[k]}' $OUT | { read -r h; echo "$h"; sort; }
```

filosofos/go.mod

```
module filosofos

go 1.21
```